

AEM Messaging Connection

Module `srv/messaging/aemessaging.ts` — pulls all five AEM settings from the SAP Credential Store, then opens a SAP Advanced Event Mesh (Solace) session and binds a consumer to the employee queue.

CollabSphere — SAP CAP App | Broker: SAP Advanced Event Mesh (`solclientjs`) | Queue `Q.EMPLOYEE.COLLABSPHERE`

1. What this module does

1. Reads `AEM_HOST`, `AEM_VPN`, `AEM_USERNAME`, `AEM_PASSWORD`, `AEM_QUEUE` from the Credential Store (namespace `Collabsphere`, all type **Key**).
2. Initialises the Solace SDK and creates a session with those values.
3. On connect, binds a `MessageConsumer` to the queue with client acknowledgement and logs every lifecycle event.

Called once as `await aemCreateSession()` from `collabSphereService.init()`. Credential retrieval is covered in `srv/credentialStore/CredentialStore_Explained.pdf`.

2. Full source — `aemessaging.ts`

```
1 import solace from "solclientjs";
2 import { readCredentialValue } from "../credentialStore/credentialStore.js";
3
4 const AEM_CREDSTORE_NAMESPACE =
5   process.env.AEM_CREDSTORE_NAMESPACE ?? "Collabsphere";
6
7 // Every AEM setting is stored in the SAP Credential Store as a "key" credential
8 // in the AEM_CREDSTORE_NAMESPACE namespace, so it is read from the /key path.
9 async function readAemKey(name: string): Promise<string> {
10   return readCredentialValue(name, AEM_CREDSTORE_NAMESPACE, "key");
11 }
12
13 async function aemCreateSession() {
14   // All AEM credentials come from the SAP Credential Store; a failed read stops
15   // startup (no env-var fallback).
16   const [host, vpn, username, password, queue] = await Promise.all([
17     readAemKey("AEM_HOST"),
18     readAemKey("AEM_VPN"),
19     readAemKey("AEM_USERNAME"),
20     readAemKey("AEM_PASSWORD"),
21     readAemKey("AEM_QUEUE"),
22   ]);
23
24   try {
25     // Validate credentials
26     if (!host || !vpn || !username || !password || !queue) {
27       throw new Error(
28         "Missing required AEM credentials in the Credential Store namespace " +
29         `${AEM_CREDSTORE_NAMESPACE}.`,
30       );
31     }
32
33     // Solace-compatible logger
34     const logger = new solace.LogImpl(
35       console.trace.bind(console),
```

```

36     console.debug.bind(console),
37     console.info.bind(console),
38     console.warn.bind(console),
39     console.error.bind(console),
40     console.error.bind(console),
41 );
42
43 // Initialize Solace JavaScript API
44 solace.SolclientFactory.init({
45     profile: solace.SolclientFactoryProfiles.version10,
46     logLevel: solace.LogLevel.INFO,
47     logger: logger,
48 });
49
50 console.log("Solace SDK initialized.");
51
52 // Create Solace session
53 const session = solace.SolclientFactory.createSession({
54     url: host,
55     vpnName: vpn,
56     userName: username,
57     password: password,
58 });
59
60 console.log("Solace session created.");
61
62 // Successful connection
63 session.on(solace.SessionEventCode.UP_NOTICE, () => {
64     console.log("Successfully connected to AEM!");
65
66     const queueDescriptor = new solace.QueueDescriptor({
67         name: queue,
68         type: solace.QueueType.QUEUE,
69     });
70
71     console.log(`Queue descriptor created: ${queue}`);
72
73     // Create message consumer bound to the queue
74     const messageConsumer = session.createMessageConsumer({
75         queueDescriptor: queueDescriptor,
76         acknowledgeMode: solace.MessageConsumerAcknowledgeMode.CLIENT,
77     });
78
79     console.log("Message consumer created.");
80
81     // Queue connected successfully
82     messageConsumer.on(solace.MessageConsumerEventName.UP, () => {
83         console.log(`Successfully connected to queue: ${queue}`);
84     });
85
86     // Queue connection failed
87     messageConsumer.on(
88         solace.MessageConsumerEventName.DOWN_ERROR,
89         (error) => {
90             console.error("Queue connection failed:", error);
91         },
92     );
93
94     // This consumer became the single active flow on the (Exclusive) queue
95     messageConsumer.on(solace.MessageConsumerEventName.ACTIVE, () => {
96         console.log(`Consumer is ACTIVE on queue: ${queue}`);
97     });

```

```

98
99 // Another bound consumer took over as the active flow; this one is idle
100 messageConsumer.on(solace.MessageConsumerEventName.INACTIVE, () => {
101   console.log(`Consumer is INACTIVE on queue: ${queue}`);
102 });
103
104 // Message received
105 messageConsumer.on(solace.MessageConsumerEventName.MESSAGE, (message) => {
106   const payload = message.getBinaryAttachment()?.toString();
107   console.log("Message received:", payload);
108   message.acknowledge();
109 });
110
111 // Connect consumer to queue
112 console.log(`Connecting to queue: ${queue}`);
113
114 messageConsumer.connect();
115 });
116
117 // Connection failure
118 session.on(solace.SessionEventCode.CONNECT_FAILED_ERROR, (error) => {
119   console.error("Failed to connect to AEM:", error);
120 });
121
122 // Disconnection
123 session.on(solace.SessionEventCode.DISCONNECTED, () => {
124   console.log("Disconnected from AEM.");
125 });
126
127 // Connect to AEM
128 console.log("Connecting to AEM...");
129 session.connect();
130 } catch (error: any) {
131   console.error("Error connecting to AEM messaging service:", error.message);
132 }
133 }
134
135 export { aemCreateSession };

```

3. Code walkthrough

Imports & namespace (lines 1–5)

`solclientjs` is the Solace client. `readCredentialValue` comes from the sibling credential-store module. `AEM_CREDSTORE_NAMESPACE` is the only environment variable left — a namespace *name*, not a secret — defaulting to `"Collabsphere"`.

`readAemKey()` (lines 9–11)

Thin wrapper: every AEM credential is a **Key** credential, so this always passes `kind: "key"`. `readCredentialValue` base64-decodes the "key" value before returning it.

Parallel credential load (lines 16–22)

`Promise.all([ ... ])` fires the five reads concurrently — five independent mTLS GET + JWE round-trips — and destructures the results into `host, vpn, username, password, queue`. If any read rejects, the whole function rejects and startup stops (there is no env-var fallback).

Validation (lines 25–31)

Guards against a credential that exists but is empty; the message names the namespace so a missing entry is easy to locate in the cockpit.

Solace init (lines 33–50)

`solace.LogImpl` routes SDK logs through `console`. `SolclientFactory.init({ profile: version10, LogLevel: INFO })` must run once before any session is created.

Session (lines 52–60)

```
solace.SolclientFactory.createSession({
  url:      host,          // wss:// ... messaging.solace.cloud:443
  vpnName:  vpn,           // message VPN
  userName: username,
  password: password,
});
```

`createSession` only configures — the socket opens at `session.connect()` (line 129).

Session events (lines 63–129)

Event	Handler
UP_NOTICE	connected — build the <code>QueueDescriptor</code> , create the consumer, wire its events, <code>messageConsumer.connect()</code>
CONNECT_FAILED_ERROR	log the failure reason
DISCONNECTED	log the disconnect

Consumer events (lines 82–114)

Event	Meaning
UP	bound to the queue
DOWN_ERROR	bind failed
ACTIVE / INACTIVE	this flow is / is not the active consumer on an exclusive queue
MESSAGE	read <code>getBinaryAttachment()</code> , log it, then <code>message.acknowledge()</code> (required — <code>acknowledgeMode: CLIENT</code>)

Error handling (lines 130–132)

Everything from validation onward is wrapped in `try/catch` that logs `error.message`. The credential reads *before* the try, so a Credential Store failure is **not** swallowed — it stops startup.

4. Connection flow

```
collabSphereService.init()
  → await aemCreateSession()
    Promise.all([ readAemKey x5 ])      → Credential Store (mTLS + JWE)
    solace.SolclientFactory.init(...)
    session = createSession({url, vpnName, userName, password})
    session.connect()
    on UP_NOTICE:
      consumer = session.createMessageConsumer({ queueDescriptor, CLIENT ack })
      consumer.connect()
        on UP:      "Successfully connected to queue: Q.EMPLOYEE.COLLABSPHERE"
        on MESSAGE: log payload → message.acknowledge()
```

5. Successful startup log

```
resolving cloud service bindings...
bound Credential_Store to cf managed service Credential_Store:cred_key
Solace SDK initialized.
Solace session created.
Connecting to AEM...
[cds] - server listening on { url: 'http://localhost:4004' }
Successfully connected to AEM!
Queue descriptor created: Q.EMPLOYEE.COLLABSPHERE
Message consumer created.
Successfully connected to queue: Q.EMPLOYEE.COLLABSPHERE
```

6. Errors seen while building this

Symptom	Cause	Fix
Credential Store read failed ... 400 Bad Request	credential type missing from the API path	read from <code>/api/v1/credentials/key</code> (all AEM creds are Key type)
Cannot read properties of undefined (reading 'asKey')	namespace import of the CommonJS <code>node-jose</code>	default import in <code>credentialStore.ts</code>
<code>[EADDRINUSE]</code> port 4004	a previous <code>cds watch</code> still running	stop the old process